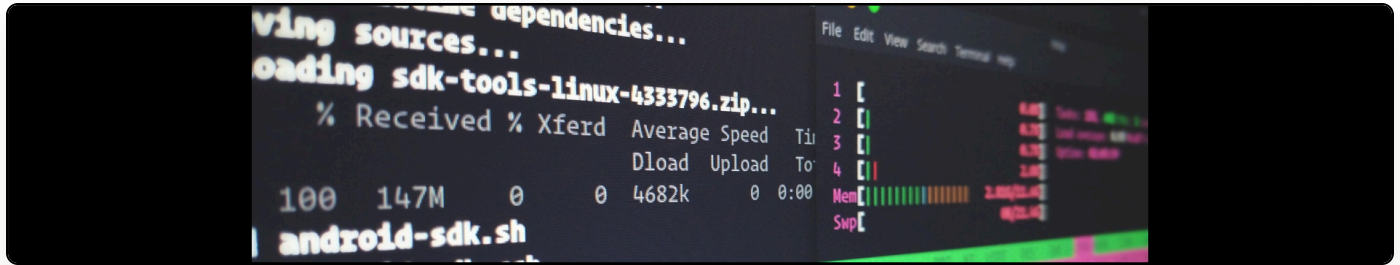


Mendeley Android Client Progress (w/ Video, Source and APK!)

Cite as: Martin Paul Eve, "Mendeley Android Client Progress (w/ Video, Source and APK!)", *eve.gd: Martin Paul Eve*, January 04, 2011, <https://doi.org/10.59348/6a9qn-21p79>.



Before anybody non-techie gets excited by the heading there, I'm not claiming this is anywhere near production-ready. In fact, it's not even functional. However, from a technical perspective, my Android Mendeley client has reached a milestone of successful OAuth login. In this post I will give some samples of the application as it stands, including an APK download, source and video, and also detail the measures I undertook to get OAuth working using Signpost under Java for Android.

First up, there's now a [Google Code project](#), which also includes GPL v3 licensed [source code](#) of my initial commit.

Secondly, here's a video of the client "in action" (running on the emulator):

Mendeley for Android OAuth Success

Martin Eve



Watch on

</param></param></param>

</embed>

Thirdly, if you felt the need to try out this "awesomeness" (read: currently useless fail) on your own device, you can [download the apk](#) and install it as an untrusted application.

As you can see, it successfully authenticates, but the API is throwing status 500 Internal Server errors on every API call at the moment. Whether this is a problem at their end, or mine, is unclear for now.

The Seriously Techie Bit

Finally, here's the technical explanation of things I tried, failed and then failed better at:

The first thing to note is that the latest version of [signpost](#) (1.2.1.1) will NOT work with Mendeley on Android. You'll certainly need 1.2. If you ignore this advice, your requests will all fail at `provider.retrieveAccessToken`.

Secondly, before calling `provider.retrieveAccessToken` you need to specify `provider.setOAuth10a(true)`, or you will end up with a response telling you that there was no consumer key provided. Even though there was.

From MendeleyConnector.java:

```
m_provider.setOAuth1a(true);  
m_provider.retrieveAccessToken(m_consumer, code);
```

Thirdly, as I mentioned in an earlier post, Mendeley will not allow you to perform a callback to any URL schema except http. For this reason, I have included a php script which must be placed on a webserver and simply performs a URL redirect when the oauth validation token is passed back in:

```
<?php  
/*  
 *  
 *  Mandroid: a Mendeley Android client  
 *  Copyright 2011 Martin Paul Eve <martin@martineve.com>  
 *  
 *  This file is part of Mandroid.  
 *  
 *  Mandroid is free software: you can redistribute it and/or modify  
 *  it under the terms of the GNU General Public License as published by  
 *  the Free Software Foundation, either version 3 of the License, or  
 *  (at your option) any later version.  
 *  
 *  Mandroid is distributed in the hope that it will be useful,  
 *  but WITHOUT ANY WARRANTY; without even the implied warranty of  
 *  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
 *  GNU General Public License for more details.  
 *  
 *  You should have received a copy of the GNU General Public License  
 *  along with Mandroid. If not, see <http://www.gnu.org/licenses/>.  
 */  
if($_GET['oauth_verifier'] != "") {  
    header( 'Location: martineve-mandroid:///?oauth_verifier=' .  
urlencode($_GET['oauth_verifier']) ) ;  
}  
?>
```

Ignore the signpost developers insistence on using the Apache Commons

OAuthProvider/Consumer, they don't work whereas DefaultOAuthProvider works fine on signpost 1.2.

If anyone is interested in all the tweaks it took to get the 3 leg phase working, I suggest looking at `MendeleyConnector.java` (heavily modified from Clemens' version to do a direct callback into the application), but various calls are also in the related activities, `MendeleyDroidLogin.java` and the `onResume` function in `MainScreenTabWidget` (which handles the callback from the browser auth). The portion that is currently failing lies in `MendeleyAPITask.java`, which will eventually be used properly as an `AsyncTask`, but is for now just running as a synchronous call from the UI thread to make debugging easier.